Lab 8 - EE 446

Part 1: Edge Impulse

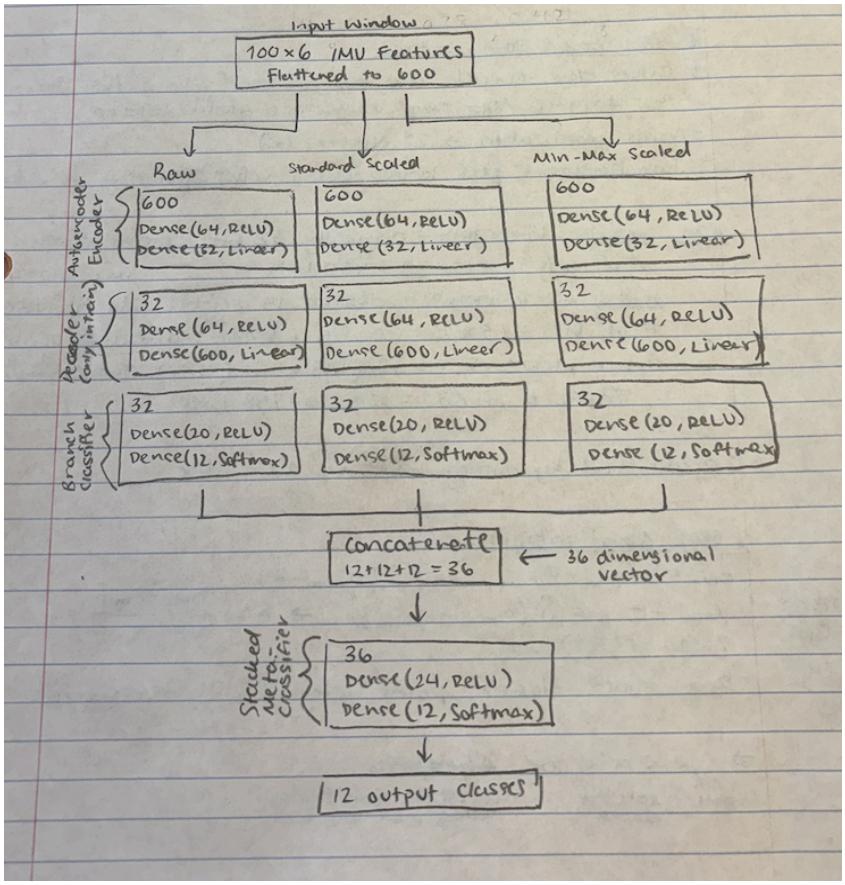
Link to Project: https://studio.edgeimpulse.com/public/1011592/live

Screenshot of Inference:

```
Sampling...
Timing: DSP 159 ms, inference 553 us, anomaly 0 ms, postprocessing 66 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 68 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 160 ms, inference 574 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
```

Part 2: Discussion Questions

Question 1: Model architecture and ensemble flow



Question 2: Pruning and QAT methodology

Pruning is applied using the TensorFlow Model Optimization Toolkit (TF-MOT) function prune_low_magnitude(). Pruning works by forcing many parameters to zero to remove the less important weights. The pruning uses a polynomial sparsity schedule, meaning that sparsity grows throughout the training process, from 10% sparsity to 80% sparsity over 500 pruning steps. Therefore the model can slowly adjust to the sparsity instead of having a very sudden change.

After this, the wrappers are removed. With the wrappers, the model size is not much decreased, so they are removed before QAT. As well, quantization libraries do not expect there to be wrappers since they are made to expect unmodified Keras models. The wrappers mainly contain pruning metadata and mask-management logic, so they are not

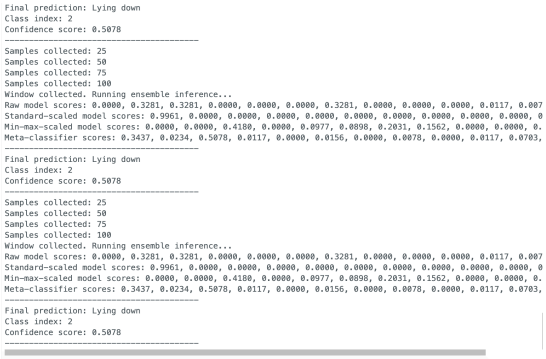
QAT is done with quantize_model(). It inserts quantization simulations so the noise that is introduced by quantization can be used during the training. Therefore when it is quantized, the weights quantized were trained to maximize accuracy with the noise.

The additional mask enforcement happens because this model is pruned. Without that QAT would not need it. Since pruning forces parameters to zero, which is what makes the model smaller, the QAT when training may grow the parameters again. By applying the pruning masks after QAT, the original pruned parameters are enforced to stay 0.

The final model is converted to an int8 TFLite model using a TFLite converter. We use a representative data generator to estimate the range of the data which will allow us to accurately quantize the model by having the data range be the optimal values, by finding the scaling factors.

When deploying to a resource constrained device, the amount of memory that can be used for holding the model and the resources for inference are much smaller than the laptop/ computer the model was built and trained on. By using pruning and int8 quantization you can get high accuracy with small memory by building and training a good model and using these models to find the best parameters to keep and making them smaller. The original accuracy of the baseline model can be well preserved by these methods.

Question 3: Arduino deployment and real-world behavior



This output is acceptable for the resting board. It is completely still and the prediction is that it is lying down. However, once I started moving it around there were only two other classes that were predicted, “Standing up” and "Knees bending”. No other classes were predicted.

“Standing up” was predicted when I moved the board at all on the horizontal axis. “Knees bending” was more when the board was moved along the vertical axis. I think this makes sense, but I thought that standing up would be predicted when the board was not moving, but above its starting position, not when actively moving.

I think that better instructions on how to use the model would help make the deployment more reliable, especially on how the board should be oriented/moved when deployed. This shows that knowledge of your dataset is as important as building the model.

Question 4: Deployment Behavior on the Arduino

There were unexpected predictions during deployment. The board mostly predicted “Standing up” but this was when the board was moving constantly. Only 3 out of the 12 classes were predicted at all. I think this is due to the fact that the board is being moved in a very different way than how the sensor that collected the training data was. I was moving the board on a cord but the data was collected most likely by a sensor connected to a person.

Other possible causes may include differences in how the board was oriented and noise on our sensor vs the original sensor. Due to these differences, the model may favor certain classes. It is possible that the movements performed during testing did not activate the feature patterns associated with the other classes.